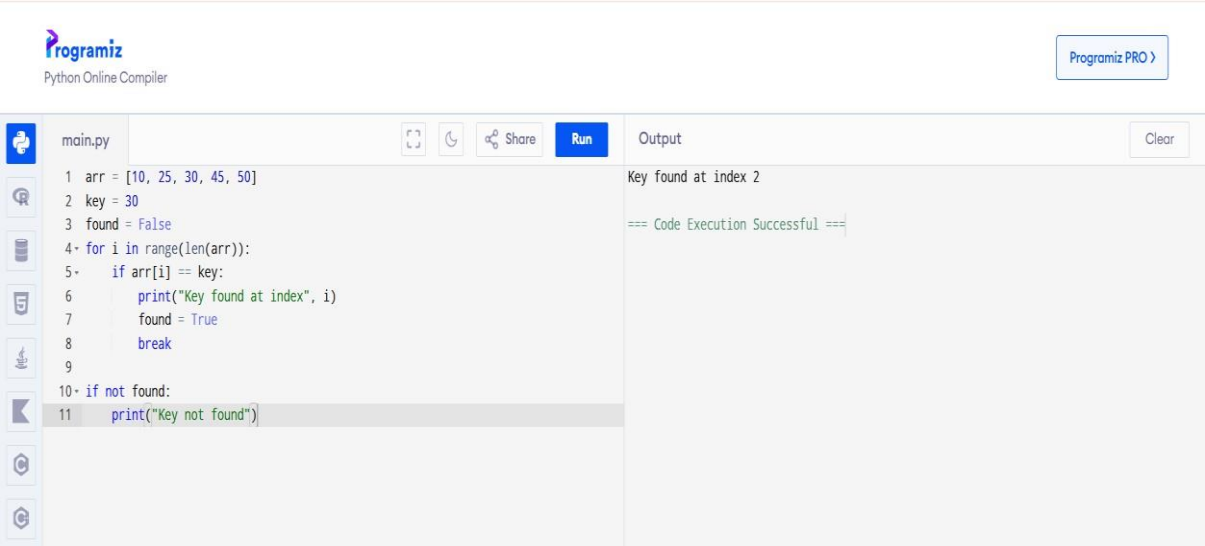
Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify BigO, Ω, Θ for sample programs

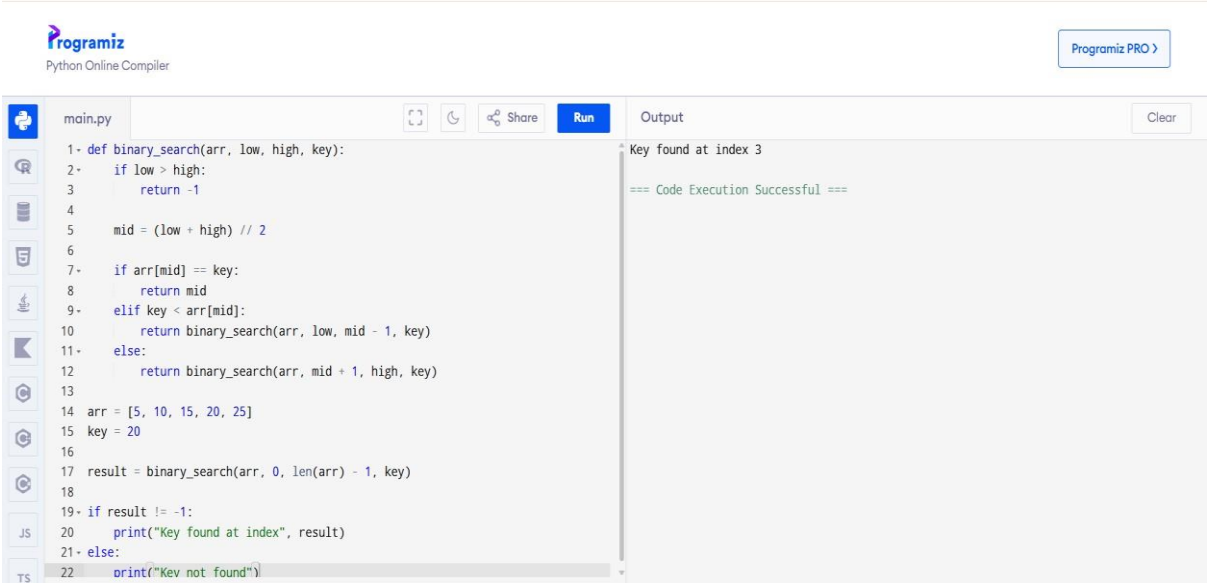
NAME:Ganji Madhan Kumar

REG NO: 192472328

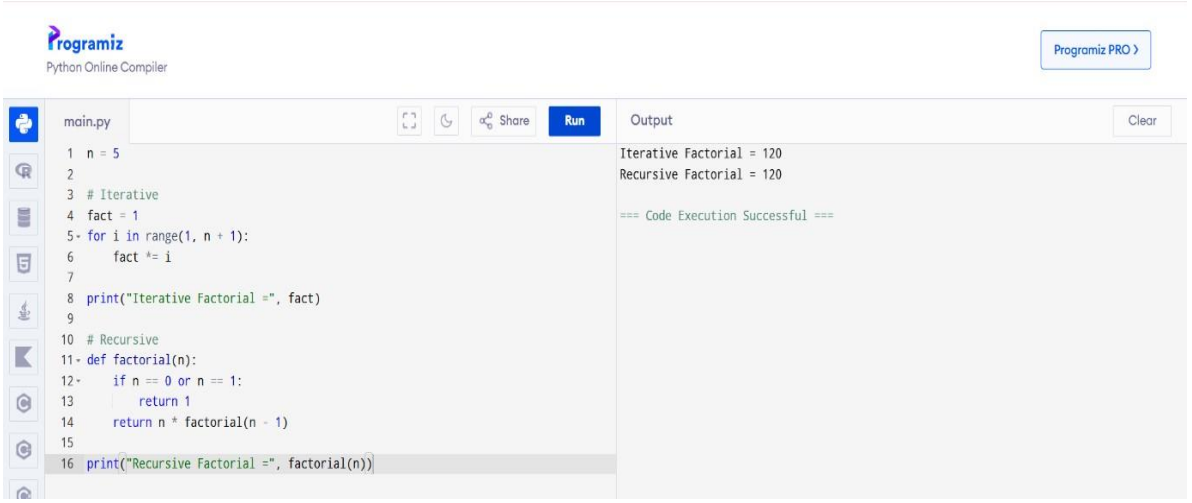
Experiment 1: Linear Search (Iterative)



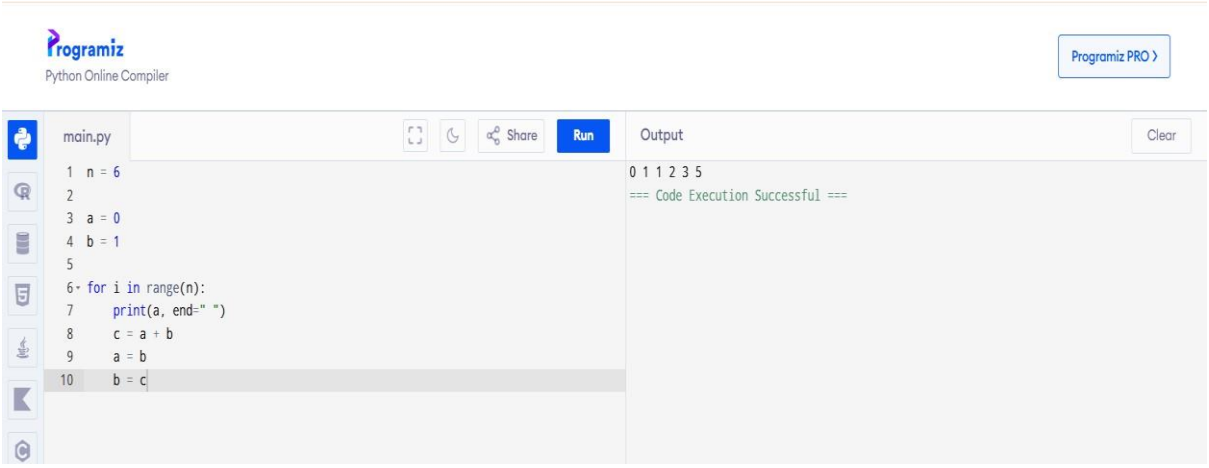
Experiment 2: Binary Search (Recursive)



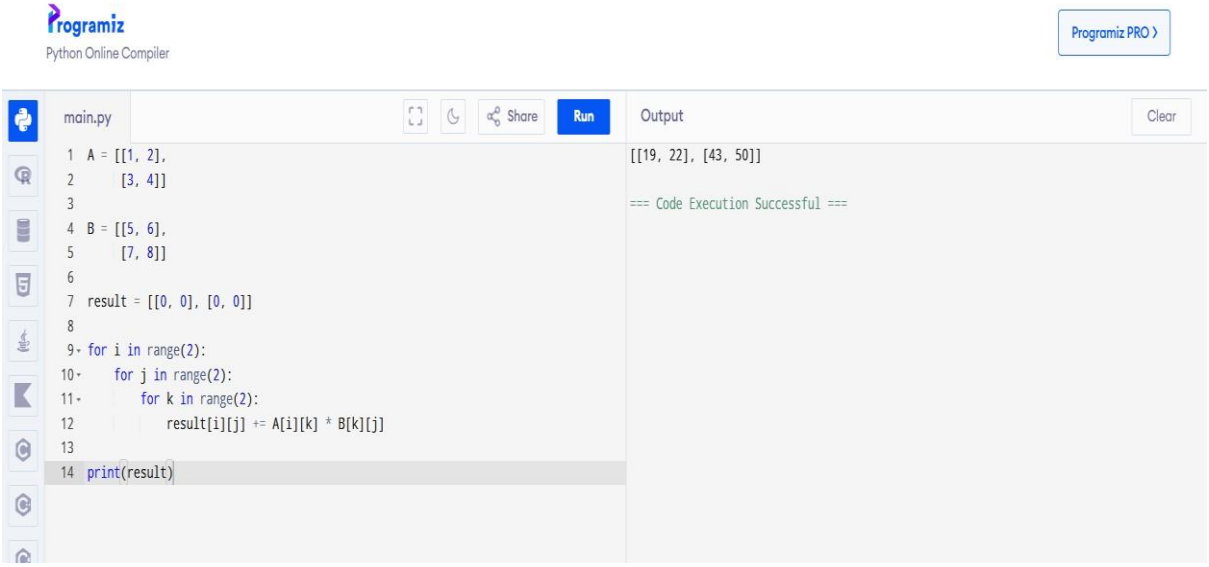
Experiment 3: Factorial (Iterative and Recursive)



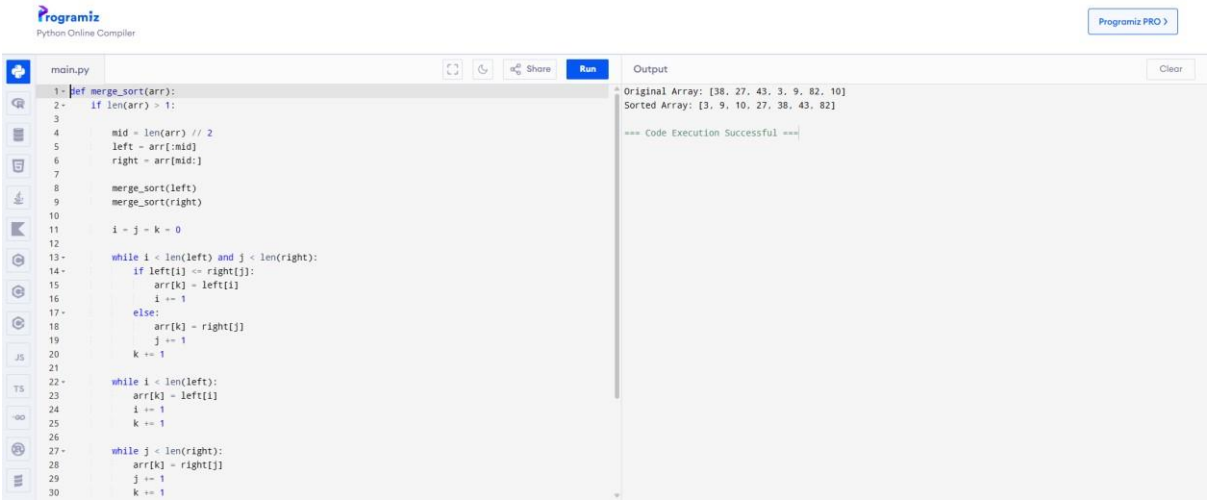
Experiment 4: Fibonacci Series



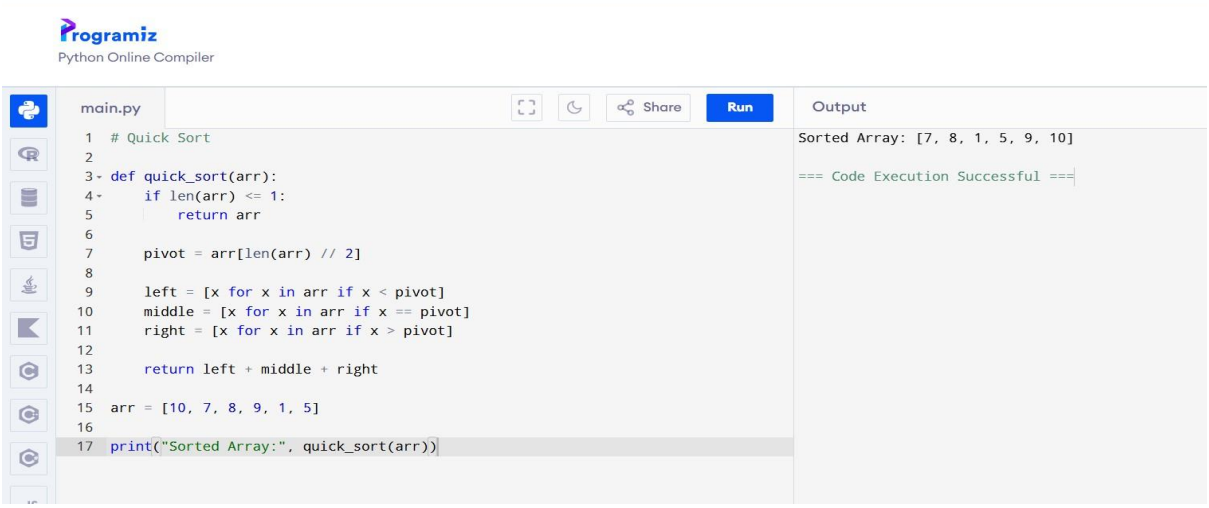
Experiment 5: Matrix Multiplication



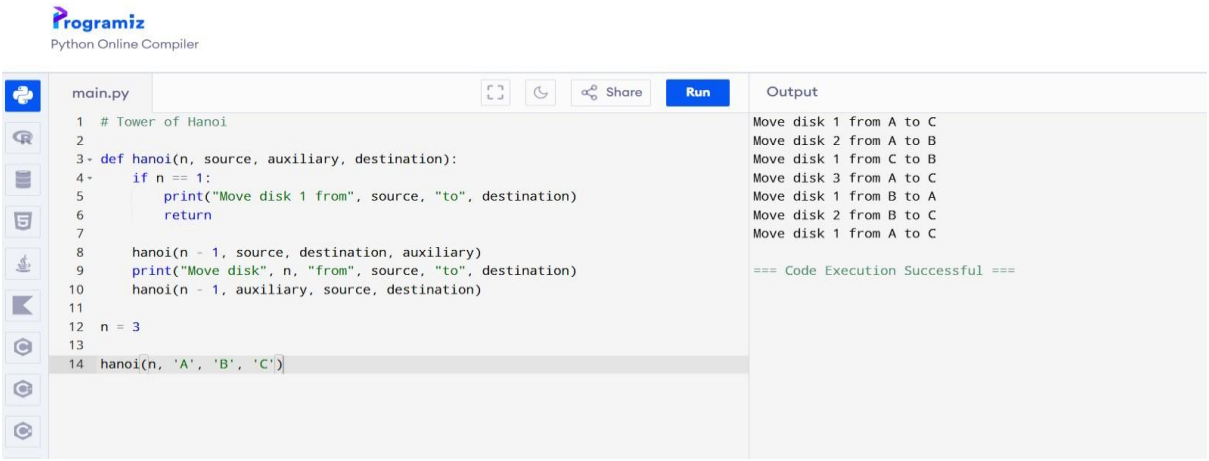
Experiment 6: Merge Sort



Experiment 7: Quick Sort



Experiment 8: Tower of Hanoi



Experiment 9: GCD (Euclidean Algorithm)

Programiz
Python Online Compiler

main.py

```python
# GCD using Euclidean Algorithm

def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a = 48
b = 18

print("GCD =", gcd(a, b))
```

Output

```
GCD = 6

=== Code Execution Successful ===
```

Experiment 10: Insertion Sort

Programiz
Python Online Compiler

main.py

```python
# Insertion Sort

arr = [12, 11, 13, 5, 6]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted Array:", arr)
```

Output

```
Sorted Array: [5, 6, 11, 12, 13]

=== Code Execution Successful ===
```

Experiment 11: Heap Sort

Programiz
Python Online Compiler

main.py

```python
# Heap Sort

import heapq

arr = [4, 10, 3, 5, 1]

heapq.heapify(arr)

sorted_list = []

while arr:
    sorted_list.append(heapq.heappop(arr))

print("Sorted Array:", sorted_list)
```

Output

```
Sorted Array: [1, 3, 4, 5, 10]

=== Code Execution Successful ===
```

Experiment 12: Counting Inversions

Programiz
Python Online Compiler

main.py

```python
# Counting Inversions (Brute Force)

arr = [2, 4, 1, 3, 5]

count = 0

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            count += 1

print("Number of Inversions:", count)
```

Output

```
Number of Inversions: 3

=== Code Execution Successful ===
```

Experiment 13: Maximum Subarray Sum (Kadane's Algorithm)

Programiz
Python Online Compiler

main.py

```python
# Maximum Subarray Sum

arr = [-2, -3, 4, -1, -2, 1, 5, -3]

max_sum = arr[0]
current_sum = arr[0]

for i in range(1, len(arr)):
    current_sum = max(arr[i], current_sum + arr[i])
    max_sum = max(max_sum, current_sum)

print("Maximum Subarray Sum =", max_sum)
```

Output

```
Maximum Subarray Sum = 7

=== Code Execution Successful ===
```

Experiment 14: Exponentiation (Fast Power)

Programiz
Python Online Compiler

main.py

```python
# Fast Exponentiation

def power(x, n):
    if n == 0:
        return 1

    half = power(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

x = 2
n = 10

print("Result =", power(x, n))
```

Output

```
Result = 1024

=== Code Execution Successful ===
```

Experiment 15: Closest Pair of Points (Brute Force)

Programiz
Python Online Compiler

main.py

Share Run

```python
# Closest Pair of Points

import math

points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_distance = float('inf')
closest_pair = ()

for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

print("Closest Pair:", closest_pair)
print("Distance =", min_distance)
```

Output

```
Closest Pair: ((1, 2), (3, 1))
Distance = 2.23606797749979

=== Code Execution Successful ===
```